

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

Ордена Трудового Красного Знамени

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«Московский технический университет связи и информатики»

Кафедра «Математическая кибернетика и информационные технологии»

Отчет по лабораторной работе №3

по дисциплине «Информационные технологии и программирование»

Выполнил: студент группы

БПИ2401

Старков Дмитрий Константинович

Проверил:

Харрасов Камиль Раисович

Москва, 2025

Содержание

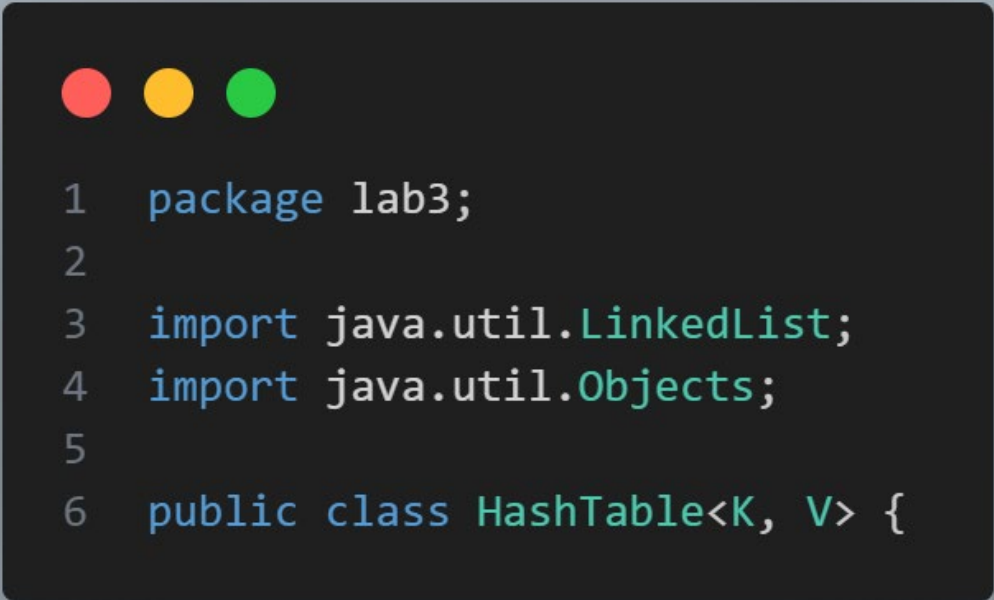
Цель работы	2
Ход работы.....	2
Вывод.....	9

Цель работы

Освоить hash-таблицы.

Ход работы

Начнем выполнение лабораторной работы с импорта необходимых библиотек и инициализации класса хеш-таблицы.

A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The code is written in Java and shows the package declaration, imports for LinkedList and Objects, and the start of a public class HashTable.

```
1 package lab3;  
2  
3 import java.util.LinkedList;  
4 import java.util.Objects;  
5  
6 public class HashTable<K, V> {
```

Здесь я импортировал LinkedList чтобы реализовать метод цепочек при использовании хэш-таблиц. А также Objects, чтобы не переопределять методы equals и hashCode.

Идем далее



```
1  private static class Entry<K, V> {  
2      private K key;  
3      private V value;  
4  
5      Entry(K key, V value) {  
6          this.key = key;  
7          this.value = value;  
8      }  
9  
10     K getKey() { return key; }  
11     V getValue() { return value; }  
12     void setValue(V value) { this.value = value; }  
13 }
```

На этом этапе я определяю внутренний класс, для удобства работы с таблицами. Этот класс будет для записей. Я делаю его публичным и статическим, чтобы доступ к его методам был в любом месте кода, без необходимости создавать объект. Этот класс я делаю типа generic, то есть ставлю <K, V>, что дает знак компилятору подставить необходимый тип при инициализации объекта. То есть я сделал код универсальным, и ключами со значениями могут быть переменные любых типов.

Далее стандартное объявление полей и конструктора, а также геттеров и сеттеров.



```
1 private LinkedList<Entry<K, V>>[] table;
2     private int capacity = 10;
3     private int size = 0;
4
5     @SuppressWarnings("unchecked")
6     public HashTable() {
7         table = new LinkedList[capacity];
8     }
9
10    @SuppressWarnings("unchecked")
11    public HashTable(int capacity) {
12        this.capacity = capacity;
13        table = new LinkedList[capacity];
14    }
```

Следующим этапом стало объявления поля `table`, которое является массивом списков `LinkedList`, включающих в себя элементы `Entry`. После чего я объявил стандартную длину массива `table` как 10, и начальный размер 0. Ниже идут конструкторы, и перед каждым из них строка `@SuppressWarnings("unchecked")`

Она нужна для того, чтобы компилятор не ругался на создание массива generic типа.

```
1  private int hash(K key) {
2      if (key == null) return 0;
3      return Math.abs(key.hashCode() % capacity);
4  }
5
6  public void put(K key, V value) {
7      int index = hash(key);
8      if (table[index] == null) {
9          table[index] = new LinkedList<>();
10     }
11
12     for (Entry<K, V> entry : table[index]) {
13         if (Objects.equals(entry.getKey(), key)) {
14             entry.setValue(value);
15             return;
16         }
17     }
18
19     table[index].add(new Entry<>(key, value));
20     size++;
21 }
```

Далее я инициализирую приватный метод hash, который включает в себе алгоритм генерации хеша. Он создаёт хеш для ключа с помощью стандартного метода hashCode, но затем берет его остаток от деления на вместимость хеш-таблицы, таким образом получая значение индекса для записи с конкретным ключом. Значение берется по модулю, потому что хэш может быть отрицательным.

После чего идет описание метода put, для добавление записи в таблицу. Он генерирует индекс с помощью ранее описанного метода hash, затем проверяет на наличие ячейку массива с этим индексом, и если в ней еще не лежит элемент LinkedList, то он создается. Потом идет цикл по всем элементам Entry в списке LinkedList в ячейке массива table по полученному ранее индексу. Этот цикл проверяет наличие переданной в параметры метода записи, и если запись с таким ключом уже существует – происходит замена

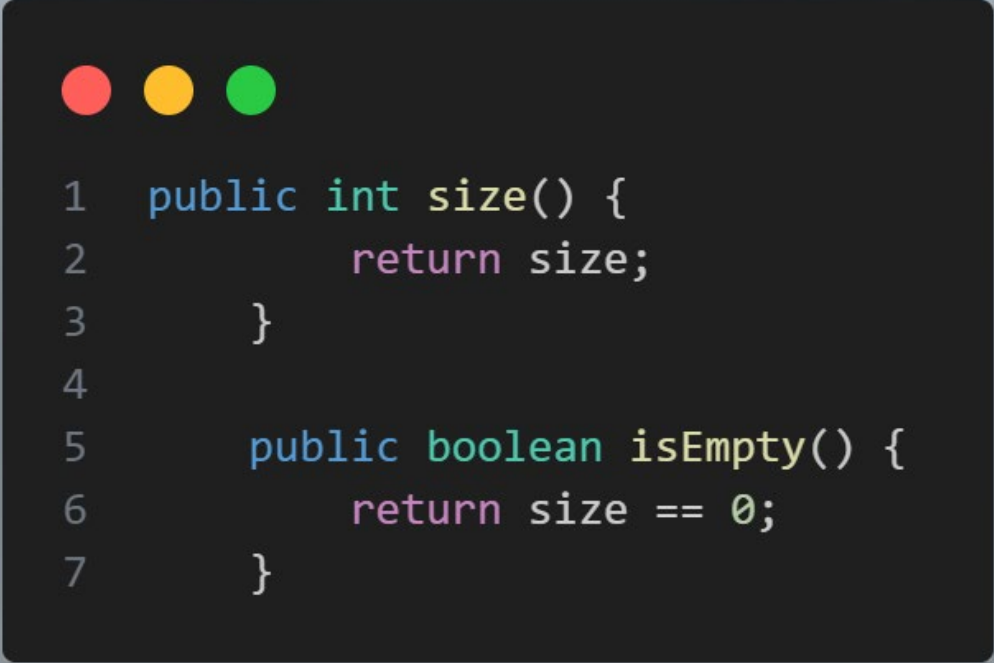
значения value и выход из метода с помощью return. Если же записи не было – она добавляется и значение размера таблицы увеличивается на 1.

```
1  public V get(K key) {
2      int index = hash(key);
3      if (table[index] == null) return null;
4
5      for (Entry<K, V> entry : table[index]) {
6          if (Objects.equals(entry.getKey(), key)) {
7              return entry.getValue();
8          }
9      }
10     return null;
11 }
12
13 public void remove(K key) {
14     int index = hash(key);
15     if (table[index] == null) return;
16
17     var iterator = table[index].iterator();
18     while (iterator.hasNext()) {
19         Entry<K, V> entry = iterator.next();
20         if (Objects.equals(entry.getKey(), key)) {
21             iterator.remove();
22             size--;
23             return;
24         }
25     }
26 }
```

Следующий метод get. Действует по схожему алгоритму: генерирует индекс, если нет записи – на выходе null, есть – выводится соответствующая запись. В методе remove я использую объект iterator для удобства работы с массивом. Var перед итератором автоматически заменяется на нужный тип Iterator<Entry<K, V>>, это сокращает код и делает его красивее. Далее идет цикл, который выполняется до тех пор, пока указатель в итераторе не дойдет до последнего объекта.

В самом цикле идет присвоение переменной entry значение записи и далее идет проверка переданного в метод ключа и ключа, который находится в

entry. Данная проверка происходит с помощью метода equals, который находится в импортированном ранее Objects. Если ключи совпадают — iterator.remove удаляет соответствующую запись и уменьшает размер таблицы на 1.



```
1  public int size() {  
2      return size;  
3  }  
4  
5  public boolean isEmpty() {  
6      return size == 0;  
7  }
```

Ну и последние 2 метода, тут комментировать особо нечего. Один выводит размер таблицы, второй — пустая она или нет.

Перехожу к индивидуальному заданию. Текст задания

«Реализация хэш-таблицы для хранения информации о студентах. Ключом является номер зачетной книжки, а значением — объект класса Student, содержащий поля имя, фамилия, возраст и средний балл. Необходимо реализовать операции вставки, поиска и удаления студента по номеру зачетки»

Для начала реализую сам класс студента.

```
1  public class Student {
2      private String firstName;
3      private String lastName;
4      private int age;
5      private double gpa;
6
7      public Student(String firstName, String lastName, int age, double gpa) {
8          this.firstName = firstName;
9          this.lastName = lastName;
10         this.age = age;
11         this.gpa = gpa;
12     }
13
14     public String getFirstName() { return firstName; }
15     public String getLastName() { return lastName; }
16     public int getAge() { return age; }
17     public double getGpa() { return gpa; }
18
19     @Override
20     public String toString() {
21         return firstName + " " + lastName + ", age: " + age + ", GPA: " + gpa;
22     }
23 }
```

Тут все стандартно, комментировать особо нечего, за исключением переопределения метода `toString`, который теперь будет выводить информацию в более читаемом формате при использовании `println`. Теперь, пользуясь ранее составленным классом `HashTable` и только что созданным классом `Student`, в отдельном файле `Main` я создам нужную хеш-

таблицу и продемонстрирую основной функционал.

```
1 public class Main {
2     public static void main(String[] args) {
3
4         HashTable<String, Student> students = new HashTable<>();
5
6         students.put("1001", new Student("Ivan", "Ivanov", 20, 4.5));
7         students.put("1002", new Student("Anna", "Petrova", 19, 4.8));
8         students.put("1003", new Student("Petr", "Sidorov", 21, 4.2));
9
10        System.out.println("Student 1001: " + students.get("1001"));
11
12        System.out.println("Total students: " + students.size());
13
14        students.remove("1002");
15        System.out.println("After removal, total students: " + students.size());
16
17        Student s = students.get("1002");
18        System.out.println("Student 1002: " + (s == null ? "not found" : s));
19
20        students.put("1004", new Student("Elena", "Smirnova", 22, 4.7));
21        System.out.println("Student 1004: " + students.get("1004"));
22        System.out.println("Total students: " + students.size());
23    }
24 }
```

Итак, в начале создаю инициализирую функцию мэйн, потом создаю хеш таблицу students.

Ну и дальше, чтобы не объяснять каждое свое действие - я просто покажу вывод.

```
Student 1001: Ivan Ivanov, age: 20, GPA: 4.5
Total students: 3
After removal, total students: 2
Student 1002: not found
Student 1004: Elena Smirnova, age: 22, GPA: 4.7
Total students: 3
```

Вывод

В ходе лабораторной работы мной были освоены хеш-таблицы, а также их организация с помощью метода цепочек.

Ссылка на гит - <https://github.com/BestStarProgrammer/IT-P/tree/main/lab2>